ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

Костюк Ю. Л.

МЕТОДЫ КОМПИЛЯЦИИ

Лекция 8

Рекурсивный спуск

Вариант анализа сверху-вниз

Преобразование грамматики для рекурсивного спуска. Используются метасимволы, как для регулярных выражений:

- вертикальная черта «или» **|**,
- скобки **{ }**,
- скобки - транзитивное замыкание **{ }***.

При этом надо устранить левую рекурсию!

Пример. Грамматика простых арифметических выражений :

$$S \to S + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (S) \mid a$$

1. Дополнение грамматики начальным правилом: $S_0 \to S \perp$

2. Преобразованная грамматика:

$$S_0 \to S \perp$$

$$S \to T \{+ T\}^*$$

$$T \to F \{* F\}^*$$

$$F \to (S) \mid a$$

S_0 - новый начальный нетерминал

Анализатор рекурсивного спуска – набор рекурсивных функций

Глобальные переменные:

C – массив (строка) входных символов (лексем);

i – номер текущего входного символа.

Для каждого нетерминала – своя функция, которая анализирует порождение этого нетерминала.

Перед вызовом функции: C[i] – ***первый*** символ порождения нетерминала.

После работы функции: C[i] – ***первый*** символ после порождения нетерминала (если не было ошибки).

Результат функции: true (нет ошибки) или false (ошибка, тогда C[i] – ошибочный символ).

Пример.

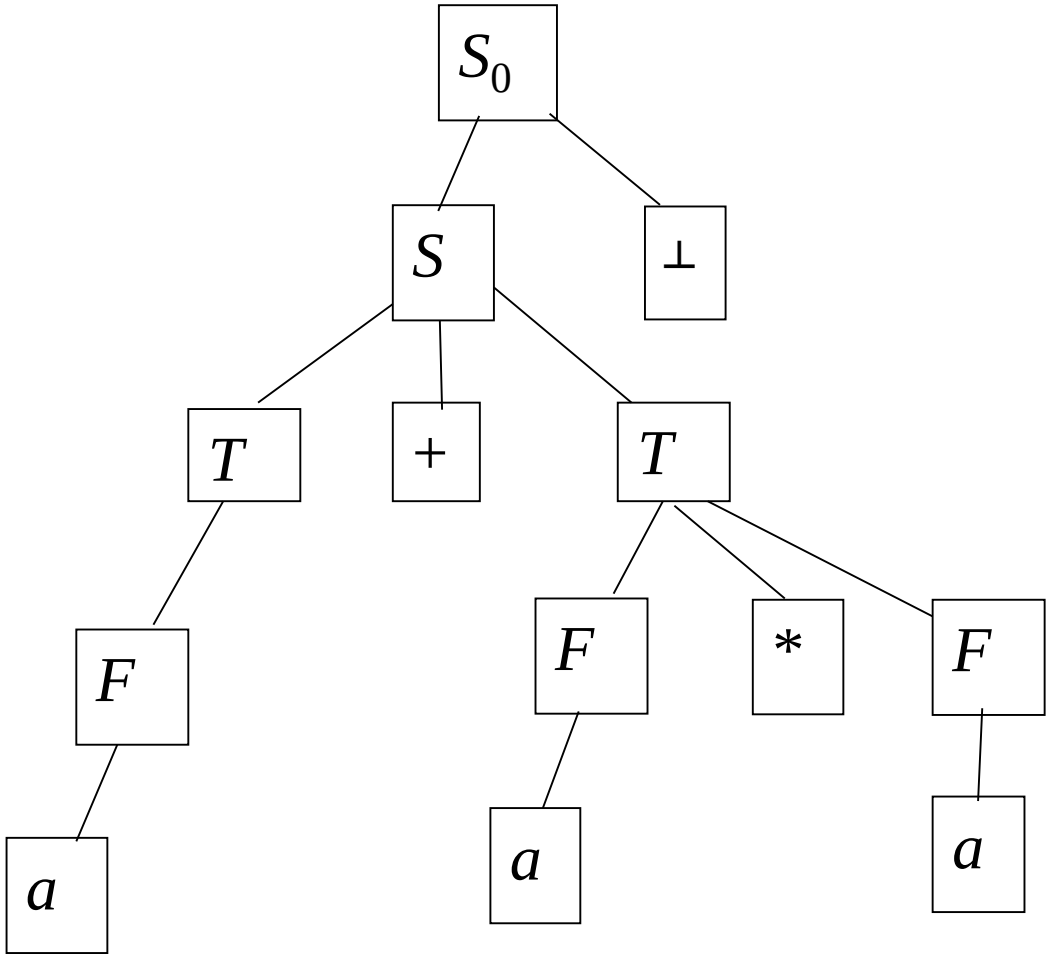
```
function S0:boolean;
begin
  if S then begin
    if C[i]='⊥' then begin
      i:=i+1; S0:=true
    end
    else S0:=false
  else S0:=false
end;
```

```
function S:boolean;
  var b:boolean;
begin b:=T;
  while b and (C[i]='+') do
    begin i:=i+1; b:=T end;
  S:=b
end;
```

```
function T:boolean;
  var b:boolean;
begin b:=F;
  while b and (C[i]='*') do
    begin i:=i+1; b:=F end;
  T:=b
end;
```

```
function F:boolean;
begin
  if C[i]='(' then begin i:=i+1;
    if S and (C[i]=')') then
      begin i:=i+1; F:=true end
    else F:=false;
  end
  else if C[i]='a' then
    begin i:=i+1; F:=true end
  else F:=false;
end;
```

На рисунке изображено неявно строящееся при разборе дерево порождения цепочки: $a + a * a \perp$



Пример. Грамматика простых арифметических выражений с устраненной левой рекурсией, после преобразования к нестрогой нормальной форме Грейбах и дополнения грамматики начальным правилом:

$$S_0 \to S \perp$$

$$S \to (S)VU \mid aVU$$

$$U \to + TU \mid \lambda$$

$$T \to (S)V \mid aV$$

$$V \to * FV \mid \lambda$$

$$F \to (S) \mid a$$

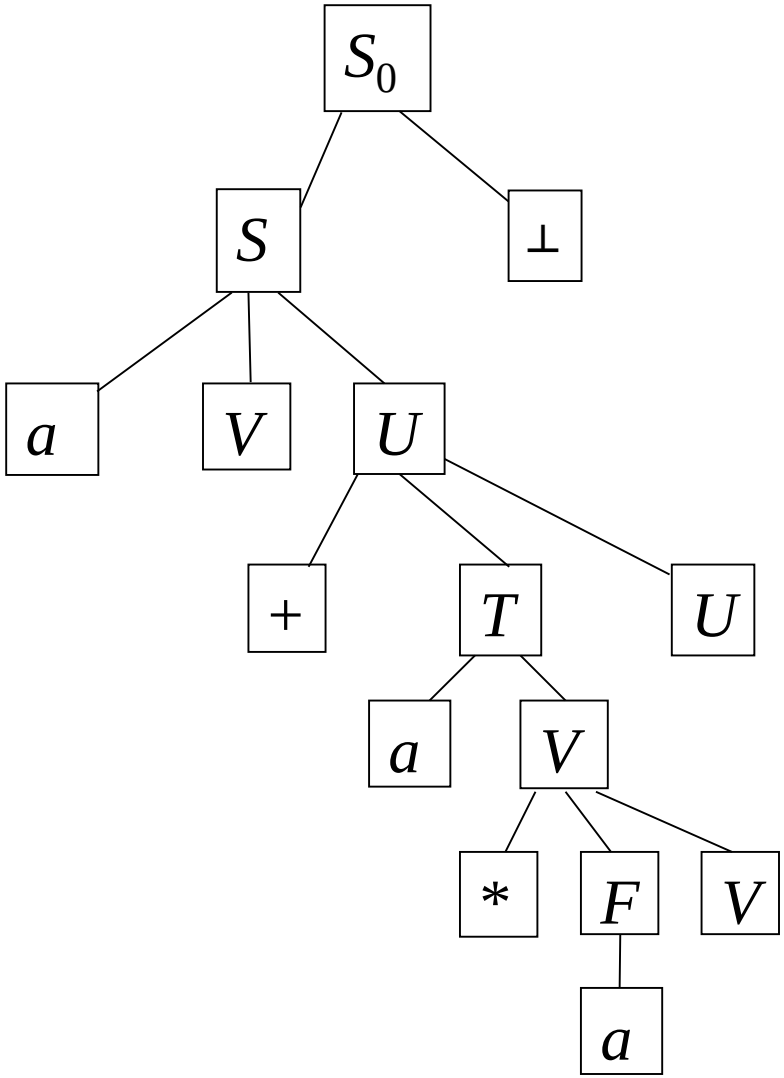
```
function S:boolean;
  var b:boolean;
begin
  if C[i]='(' then begin i:=i+1;
    if S and (C[i]=')') then
      begin i:=i+1; b:=true end
    else b:=false;
  end
  else if C[i]='a' then
    begin i:=i+1; b:=true end
  else b:=false;
  if b then b:=V;
  if b then b:=U;
  S:=b
end;
```

```
function T:boolean;
  var b:boolean;
begin
  if C[i]='(' then begin i:=i+1;
    if S and (C[i]=')') then
      begin i:=i+1; b:=true end
    else b:=false;
  end
  else if C[i]='a' then
    begin i:=i+1; b:=true end
  else b:=false;
  if b then b:=V;
  T:=b
end;
```

```
function U:boolean;
  var b:boolean;
begin b:=true;
  if C[i]='+' do begin
    i:=i+1; b:=T;
    if b then b:=U;
  end;
  U:=b
end;
```

```
function V:boolean;
  var b:boolean;
begin b:=true;
  if C[i]='*' do begin
    i:=i+1; b:=F;
    if b then b:=V;
  end;
  V:=b
end;
```

На рисунке изображено неявно строящееся при разборе дерево порождения цепочки: $a + a * a \perp$



СИНТАКСИЧЕСКИЙ АНАЛИЗ СНИЗУ-ВВЕРХ

Дерево порождения для КС-грамматики

Все порождающие правила имеют вид: $A \rightarrow \gamma$,

где A – нетерминальный символ, γ – цепочка из терминальных и нетерминальных символов.

Процесс порождения в КС-грамматике – построение дерева порождения.

Вершины дерева – терминальные и нетерминальные символы.

Корень дерева – начальный нетерминал, вниз от него идут ребра, в концах которых размещаются символы правой части порождающего правила.

От каждой вершины – нетерминала достраиваются вниз ребра с вершинами – символами в правой части примененного порождающего правила.

Процесс продолжается до тех пор, пока среди висячих вершин дерева не останется ни одного нетерминала.

Обход дерева слева направо по висячим вершинам дерева будет цепочкой языка, получившейся в процессе порождения.

Правое каноническое порождение – если на каждом очередном шаге порождения дерево достраивается от самой правой висячей вершины – нетерминала.

Каждому построенному дереву порождения однозначно соответствует некоторое правое каноническое порождение.

Дерево порождения можно строить от висячих вершин к корню – снизу-вверх. На очередном шаге построения к нескольким самым левым корневым вершинам поддеревьев достраиваются ребра и объединяющая их корневая вершина следующего уровня.

Каждому частично построенному дереву способом снизу-вверх соответствует недостроенная до полного дерева часть. Эта часть дерева строится правым каноническим порождением.

Автомат с магазинной памятью (МПА)

МПА задается семеркой множеств:

$$\{\Sigma, Q, q_0, F, \Gamma, Z_0, \delta\},$$

где Σ – множество (алфавит) входных символов;

Q – множество состояний МПА;

q_0 – начальное состояние, $q_0 \subset Q$;

F – множество заключительных состояний, $F \subseteq Q$;

Γ – множество (алфавит) магазинных символов;

Z_0 – начальный магазинный символ, $Z_0 \subset \Gamma$;

δ – множество правил перехода.

Действие в правиле перехода определяется входным символом, верхним символом магазина и текущим состоянием. Действие может содержать в различных сочетаниях такие шаги: а) удаление из магазина верхнего символа, б) запись в магазин нескольких символов, в) переход к чтению следующего входного символа, г) изменение текущего состояния.

запись правила перехода в виде:

$$(a, B, q_i) \rightarrow (\lambda, \gamma B, q_j),$$

где $a \subset \Sigma,\ q_i \subset Q,\ q_j \subset Q$, символ B и символы из цепочки γ все принадлежат алфавиту Г, означает, что в магазин, содержащий верхний символ B, записывается цепочка символов γ, текущее состояние становится q_j , после чего делается переход к чтению следующего входного символа. Правило:

$$(a, B, q_i) \rightarrow (a, \lambda, q_j),$$

означает, что из магазина удаляется верхний символ B, текущее состояние становится q_j , переход к чтению следующего входного символа не производится.

До начала функционирования МПА в магазине имеется начальный магазинный символ, МПА находится в начальном состоянии.

На каждом шаге работы читается очередной символ входной цепочки, начиная с первого, читается верхний символ магазина, и выполняются действия по такому правилу перехода, которое соответствует этим символам и текущему состоянию.

Работа МПА заканчивается, когда не будет ни одного подходящего правила, чтобы можно было продолжать работу.

Если при этом выполнены три условия: цепочка прочитана вся, магазин пуст, МПА находится в заключительном состоянии, то цепочка считается распознанной.

Если же хотя бы одно из этих условий нарушено, цепочка считается нераспознанной. Цепочка считается нераспознанной и в том случае, если МПА никак не может остановиться, т.е. когда он зациклил, и нет продвижения по входной цепочке.

МПА будет детерминированным (ДМПА), если на каждом шаге возможно применение не более одного правила перехода.

Если же на каком-либо шаге можно выполнить действия по двум или более правилам перехода, то МПА будет недетерминированным (НМПА), тогда он должен выполнять действия в соответствии со всеми этими правилами параллельно и независимо, создавая собственные копии.

Если хотя бы одна из копий НМПА ***распознает*** входную цепочку, то считается, что входная цепочка ***распознана*** недетерминированным МПА.

Входная цепочка считается ***нераспознанной***, если ни одна из копий МПА ***не распознала*** цепочку.

LR-разбор КС-грамматики автоматом с магазинной памятью

LR-разбор – цепочка символов читается слева направо, дерево строится снизу-вверх, ***дополняя*** правое каноническое порождение.

Пусть задана КС-грамматика. Построим по ней МПА, выполняющий грамматический LR-разбор.

Множество входных символов Σ в МПА будет совпадать с множеством терминальных символов грамматики.

Множество состояний Q будет состоять из начального состояния q_0 и заключительного q_1.

Множество магазинных символов Γ будет содержать все терминальные и нетерминальные символы грамматики и особый начальный магазинный символ ⊥:

$$\Gamma = \Sigma \cup N \cup \{\perp\}.$$

Правила перехода δ для МПА, моделирующего разбор снизу-вверх, будем строить следующим образом.

Для каждого порождающего правила вида $A \to \gamma$ будет правило перехода:

$$(\lambda, \gamma, q_0) \to (\lambda, A, q_0), \qquad (*)$$

здесь ***основа*** (правая часть правила) γ, записанная в магазине, заменяется (сворачивается) на нетерминал A.

Для каждого терминального символа a правило перехода:

$$(a, \lambda, q_0) \to (\lambda, a, q_0), \qquad (**)$$

здесь очередной входной символ a переносится в магазин.

Особое правило (завершающее):

$$(\lambda, \perp S, q_0) \to (\lambda, \lambda, q_1), \qquad (***)$$

где S – начальный нетерминал.

Такой алгоритм называют алгоритмом «перенос – свертка».

В общем случае такой МПА будет недетерминированным, и будет пытаться строить все варианты сворачивания самой левой правой части какого-либо правила, сформированного в магазине.

Теорема.

НМПА, реализующий LR-разбор некоторой КС-грамматики, допускает входную цепочку тогда и только тогда, когда цепочка порождается этой КС-грамматикой.

Доказательство – методом от «противного»

Пример. Грамматика простых арифметических выражений:

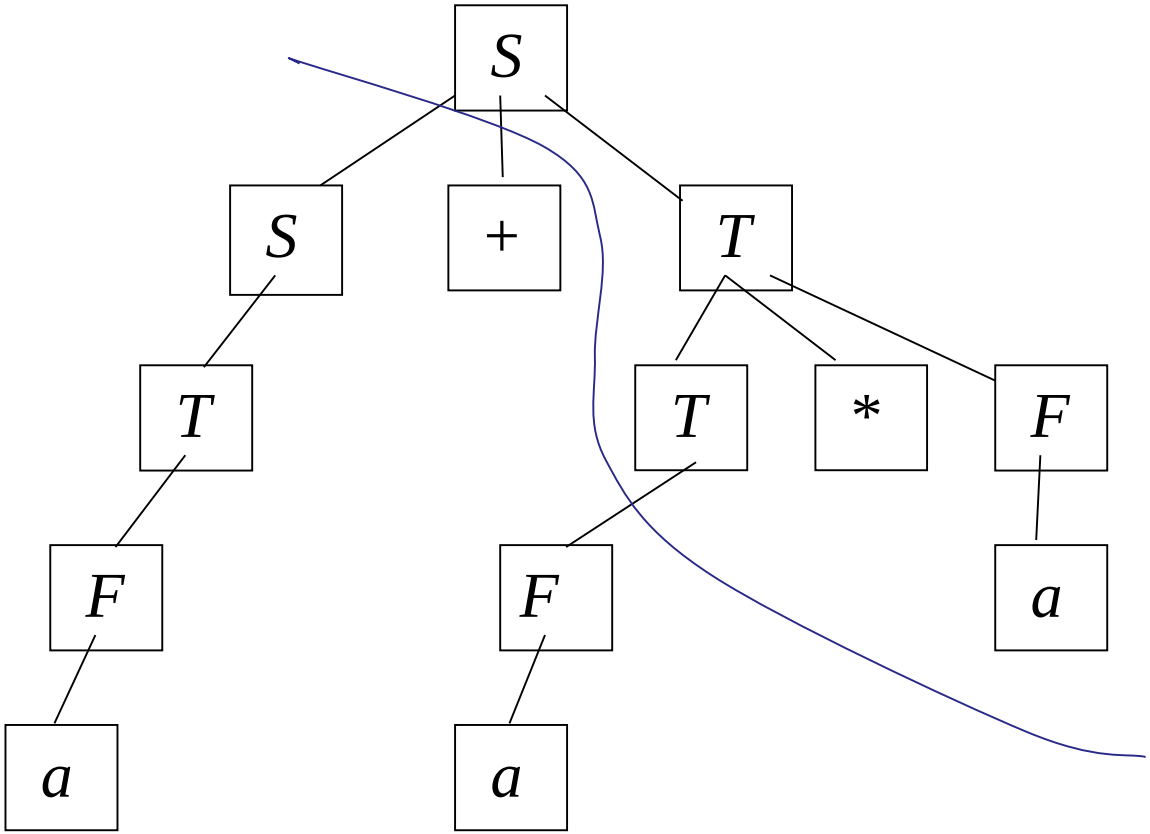
$$S \to S + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (S) \mid a$$

Пусть цепочка на входе: $a + a * a$.

В таблице представлен тот вариант шагов LR-разбора, который приводит к успешному распознаванию. Всех вариантов – бесконечно много!

№ шага	Вход	Магазин	Правило	Состояние
1	$a + a * a$	$\perp$		q_0
2	$+ a * a$	$\perp a$	$F \to a$	q_0
3	$+ a * a$	$\perp F$	$T \to F$	q_0
4	$+ a * a$	$\perp T$	$S \to T$	q_0
5	$+ a * a$	$\perp S$		q_0
6	$a * a$	$\perp S +$		q_0
7	$* a$	$\perp S + a$	$F \to a$	q_0
8	$* a$	$\perp S + F$	$T \to F$	q_0
9	$* a$	$\perp S + T$		q_0
10	a	$\perp S + T *$		q_0
11	λ	$\perp S + T * a$	$F \to a$	q_0
12	λ	$\perp S + T * F$	$T \to T * F$	q_0
13	λ	$\perp S + T$	$S \to S + T$	q_0
14	λ	$\perp S$		q_1

На рисунке изображено неявно строящееся при LR-разборе дерево порождения цепочки: *a* + *a* * *a*.



Кривая линия отделяет частично построенное дерево LR-разбором от соответствующего правого канонического порождения.

Детерминированный LR-анализ

Отношения простого предшествования

Определены для каждой пары (обозначенных как A и B) терминальных и нетерминальных символов грамматики.

- Отношение ***типа равно***, $A \doteq B$ выполняется тогда и только тогда, когда существует порождающее правило:

$$U \rightarrow \ldots A\ B \ldots$$

где U – некоторый нетерминал.

- Отношение ***типа меньше***, $A \lessdot B$ выполняется тогда и только тогда, когда существует порождающее правило:

$$U \rightarrow \ldots A\ V \ldots$$

где U ,V– некоторые нетерминалы, а из нетерминала V за один или несколько шагов выводится цепочка символов, ***начинающаяся*** с B:

$$V \Rightarrow B \ldots$$

- Отношение ***типа больше***, $A \cdot> B$ выполняется тогда и только тогда, когда существует порождающее правило:

$$U \rightarrow \ldots V W \ldots$$

где U ,V– некоторые нетерминалы,

из нетерминала V за один или несколько шагов выводится цепочка символов, ***заканчивающаяся*** на A:

$$V => \ldots A$$

B – терминальный символ,

W – либо есть В, либо из нетерминала W за один или несколько шагов выводится цепочка символов, ***начинающаяся*** с B:

$$W =>^* B \ldots$$

Функционирование анализатора, использующего *отношения простого предшествования*

1. До начала работы в магазине имеется символ $\perp$.
2. Анализируемая цепочка заканчивается символом $\perp$.
3. Для любого символа A (терминала или нетерминала грамматики) выполняются отношения предшествования:

$$\perp \lessdot A \qquad \text{и} \qquad A \gtrdot \perp$$

4. На очередном шаге работы проверяется отношение между верхним символом в магазине A и входным символом B:

   - если $A \lessdot B$ или $A \doteq B$, то B записывается в магазин;

   - если $A \gtrdot B$ то верхние символы в магазине, образующие основу с символом A в конце, сворачиваются, т.е. заменяются на нетерминал.

5. Если вся анализируемая цепочка может быть свернута к начальному нетерминалу S, то на последнем шаге работы в магазине будут символы: $\perp S$, а на входе – символ $\perp$.

Грамматика простого предшествования

КС- грамматика, в которой:

1. Для каждой пары терминальных и нетерминальных символов грамматики существует ***не более одного*** из трех возможных отношений предшествования.
2. Грамматика ***обратима***, т.е. среди всех порождающих правил нет никаких двух одинаковых правых частей.

Грамматика простого предшествования однозначна, так как анализатор работает детерминированно.

Пусть верхние символы магазина: $A_{i-1} A_i A_{i+1} \ . \ . \ . \ A_j$,

входной символ A_{j+1}, между которыми выполняются

отношения: $A_{i-1} < A_i = A_{i+1} = \ . \ . \ . = A_j > A_{j+1}$

Тогда сворачиваются символы основы: $A_i A_{i+1} \ . \ . \ . \ A_j$

Пример. Грамматика простых арифметических выражений:

$$S \to S + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (S) \mid a$$

1. Отношение типа равно для пар:

$S +$ $\quad + T$ $\quad T *$ $\quad * F$ $\quad (S$ $\quad S)$

2. Отношение типа меньше для пар:

+ и символ, порождаемый первым из T

* и символ, порождаемый первым из F

(и символ, порождаемый первым из S

3. Отношение типа больше для пар:

символ, порождаемый последним из S и +

символ, порождаемый последним из T и *

символ, порождаемый последним из S и)

Таблица отношений простого предшествования

	S	*T*	*F*	+	*	(	)	*a*
S				=			=	
T				>	=		>	
F				>	>		>	
+		<,=	<			<		<
*			=			<		<
(	<,=	<	<			<		<
)				>	>		>	
a				>	>		>	

Если для некоторых пар символов одновременно существуют отношения < и =, то такая грамматика называется грамматикой нестрогого предшествования.

Пример. Грамматика после преобразования - стратификации:

$$S \to S + T \mid T$$

$$T \to T_1$$

$$T_1 \to T_1 * F \mid F$$

$$F \to (S_1) \mid a$$

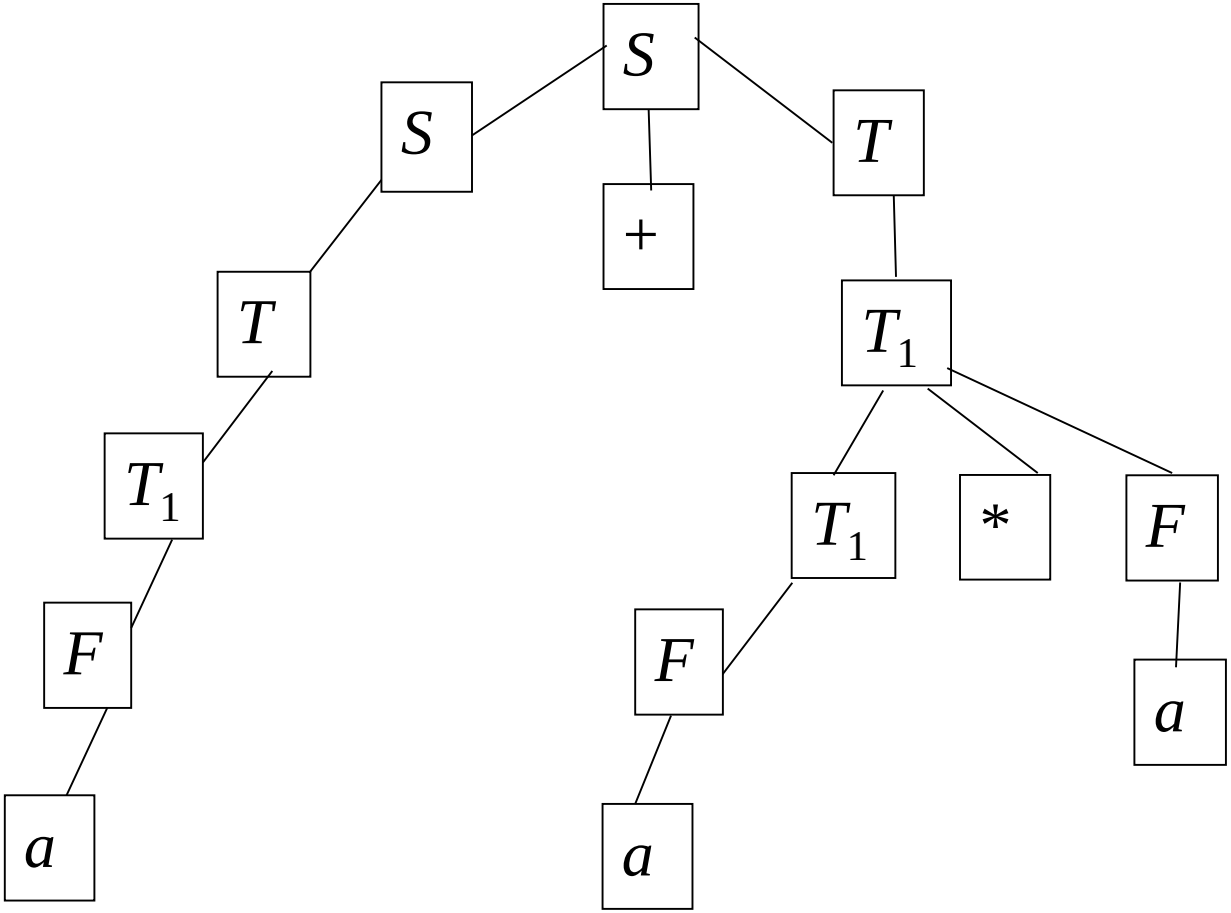
$$S_1 \to S$$

Таблица отношений преобразованной грамматики

	S	S_1	T	T_1	F	+	*	(	)	a
S						=			=	
S_1						=			=	
T						>	=		>	
T_1						>	=		>	
F						>	>		>	
+			=	<	<			<		<
*					=			<		<
(	<	=	<	<	<			<		<
)						>	>		>	
a						>	>		>	

№ шага	Вход	Магазин	Отношение	Правило
1	$a + a * a \perp$	$\perp$	$\perp < a$	$\leftarrow$
2	$+ a * a \perp$	$\perp a$	$a > +$	$F \rightarrow a$
3	$+ a * a \perp$	$\perp F$	$F > +$	$T_1 \rightarrow F$
4	$+ a * a \perp$	$\perp T_1$	$T_1 > +$	$T \rightarrow T_1$
5	$+ a * a \perp$	$\perp T$	$T > +$	$S \rightarrow T$
6	$+ a * a \perp$	$\perp S$	$S = +$	$\leftarrow$
7	$a * a \perp$	$\perp S +$	$+ < a$	$\leftarrow$
8	$* a \perp$	$\perp S + a$	$a > *$	$F \rightarrow a$
9	$* a \perp$	$\perp S + F$	$F > *$	$T_1 \rightarrow F$
10	$* a \perp$	$\perp S + T_1$	$T_1 = *$	$\leftarrow$
11	$a \perp$	$\perp S + T_1 *$	$* < a$	$\leftarrow$
12	$\perp$	$\perp S + T_1 * a$	$a > \perp$	$F \rightarrow a$
13	$\perp$	$\perp S + T_1 * F$	$F > \perp$	$T_1 \rightarrow T_1 * F$
14	$\perp$	$\perp S + T_1$	$T_1 > \perp$	$T \rightarrow T_1$
15	$\perp$	$\perp S + T$	$T > \perp$	$S \rightarrow S + T$
16	$\perp$	$\perp S$	$S > \perp$	***Выход***

На рисунке изображено неявно строящееся при разборе дерево порождения цепочки: $a + a * a \perp$



Вычисление отношений простого предшествования

Отношение ***типа равно*** элементарное, вычисляется просмотром порождающих правил, при этом формируется булевская матрица.

Для вычисления отношения $<\cdot$ определим еще одно элементарное отношение:

отношение **first** для нетерминала U и символа D, если существует порождающее правило

$$U \to D\ldots$$

Тогда, согласно определению, отношение $<\cdot$ есть произведение отношений:

$$(\dot{=})\cdot(\mathbf{first}^{+}),$$

где $\mathbf{first}^{+}$ – ***положительное*** транзитивное замыкание отношения **first**.

Вычисление отношений простого предшествования

Для вычисления отношения $\cdot>$ определим еще одно элементарное отношение:

отношение **<u>last</u>** для нетерминала U и символа D, если существует порождающее правило

$$U \rightarrow \ldots D$$

Тогда, согласно определению, отношение $\cdot>$ есть произведение отношений:

$$(\underline{\mathbf{last}}^{+})^{-1} \cdot (\doteq) \cdot (\underline{\mathbf{first}}^{*}).$$

где

<u>first</u>* – ***полное*** транзитивное замыкание отношения **<u>first</u>**,

<u>last</u>$^{+}$ – положительное транзитивное замыкание отношения **<u>last</u>**,

возведение в степень –1 обозначает вычисление обратного отношения, т.е. его транспонирование.

Использование отношения **first** для построения анализатора сверху-вниз

Отношение **first** для нетерминала U и символа D, если существует порождающее правило

$$U \to D. . .$$

Если нетерминал U ***леворекурсивный***, то элемент главной диагонали матрицы отношения **first** для нетерминала U будет равен 1.

Вычислим **first**$^+$ – *положительное* транзитивное замыкание отношения **first**.

Элемент главной диагонали матрицы отношения **first**$^+$ для нетерминала U будет равен 1 тогда и только тогда, когда имеется прямая или косвенная ***левая рекурсия*** для нетерминала U .

Отношения операторного предшествования

Определены для каждой пары (обозначенных как a и b) ***терминальных*** символов грамматики, преобразованной в ***операторную форму***, такую, что в ней нет порождающих правил вида:

$$U \rightarrow \ldots AB \ldots,$$

где U, A, B – некоторые нетерминалы.

Отношение ***типа равно***, $a \doteq b$ выполняется тогда и только тогда, когда существуют порождающие правила:

$$U \rightarrow \ldots ab \ldots \quad \text{или} \quad U \rightarrow \ldots aWb \ldots$$

где W – некоторый нетерминал.

Отношение ***типа меньше***, $a <\circ b$ выполняется тогда и только тогда, когда существует порождающее правило:

$$U \rightarrow \ldots aV \ldots$$

где U ,V– некоторые нетерминалы, а из нетерминала V за один или несколько шагов выводится цепочка символов, ***начинающаяся*** с b или нетерминала W и терминала b:

$$V \Rightarrow b \ldots \qquad \text{или} \qquad V \Rightarrow Wb \ldots$$

Отношение ***типа больше***, $a \circ> b$ выполняется тогда и только тогда, когда существует порождающее правило:

$$U \rightarrow \ldots V\, b \ldots$$

где U ,V– некоторые нетерминалы,

из нетерминала V за один или несколько шагов выводится цепочка символов, ***заканчивающаяся*** на a или нетерминал W и терминал a:

$$V \Rightarrow \ldots a \qquad \text{или} \qquad V \Rightarrow \ldots aW$$

Таким образом, нетерминальные символы ***«прозрачны»***

для отношений операторного предшествования

Замечание. В операторной грамматике при любом сворачивании основ ***никогда*** не могут появиться два рядом стоящих нетерминала.

Функционирование анализатора, использующего *отношения операторного предшествования*

1. До начала работы в магазине имеется символ $\perp$.
2. Анализируемая цепочка заканчивается символом $\perp$.
3. Для любого терминала a выполняются отношения операторного предшествования:

$$\perp \lessdot a \quad \text{и} \quad a \gtrdot \perp$$

4. На очередном шаге работы проверяется отношение между верхним терминалом в магазине a (игнорируя нетерминал, если он там есть) и входным символом b:

   - если $a \lessdot b$ или $a \doteq b$, то b записывается в магазин;

   - если $a \gtrdot b$ то верхние символы в магазине, образующие основу с символом a в конце или символом a и каким-либо нетерминалом , сворачиваются, т.е. заменяются на нетерминал, причем все ***нетерминалы*** считаются ***неразличимыми***, обозначаемыми как N.
5. Если вся анализируемая цепочка может быть свернута к начальному нетерминалу, то на последнем шаге работы в магазине будут символы: $\perp N$, а на входе – символ $\perp$.

Грамматика операторного предшествования

КС- грамматика в операторной форме, в которой:

1. Для каждой пары терминальных символов грамматики существует ***не более одного*** из трех возможных отношений предшествования.
2. Грамматика ***обратима*** с точностью до обозначений нетерминалов, т.е. среди всех порождающих правил нет никаких двух одинаковых правых частей, если все нетерминалы обозначить одним и тем же символом.

Грамматика операторного предшествования однозначна, так как анализатор работает детерминированно.

Пусть верхние символы магазина: $t_{i-1} A_i t_i A_{i+1} t_{i+1} \ldots t_j A_j t_{j+1}$,

где t – терминалы A – нетерминалы, t_{j+1} – входной символ, между которыми выполняются отношения:

$$t_{i-1} < t_i = t_{i+1} = \ldots = t_j > t_{j+1}$$

Тогда сворачиваются символы основы: $A_i t_i A_{i+1} t_{i+1} \ldots t_j A_j$

Пример. Грамматика простых арифметических выражений:

$$S \to S + T \mid T$$

$$T \to T * F \mid F$$

$$F \to (S) \mid a$$

1. Отношение типа равно для пары скобок (и).
2. Отношение типа меньше для пар:

   + и терминал, порождаемый первым из T

   * и терминал, порождаемый первым из F

   (и терминал, порождаемый первым из S
3. Отношение типа больше для пар:

   терминал, порождаемый последним из S и +

   терминал, порождаемый последним из T и *

   терминал, порождаемый последним из S и)

Таблица отношений операторного предшествования

	+	*	(	)	*a*
+	∘>	<∘	<∘	∘>	<∘
*	∘>	>	<∘	∘>	<∘
(	<∘	<∘	<∘	=	<∘
)	∘>	∘>		∘>	
a	∘>	∘>		∘>	

Вычисление отношений операторного предшествования

Кроме отношений ($\stackrel{\circ}{=}$), ($\stackrel{\bullet}{=}$), **first** и **last** определим еще два элементарных отношения:

1) отношение **firstterm** для нетерминала U и терминала d, если существует порождающее правило

$$U \to d\ldots \quad \text{или} \quad U \to Wd\ldots$$

2) отношение **lastterm** для нетерминала U и терминала d, если существует порождающее правило

$$U \to \ldots d \quad \text{или} \quad U \to \ldots dW$$

где W – некоторый нетерминал.

Тогда, согласно определению, отношение $<\circ$ есть произведение отношений:

$$(\doteq)\cdot(\underline{\mathbf{first}}^{*})\cdot(\underline{\mathbf{firstterm}}),$$

а отношение $\circ>$ есть произведение отношений:

$$((\underline{\mathbf{last}}^{*})\cdot(\underline{\mathbf{lastterm}}))^{-1}\cdot(\doteq).$$

где $\underline{\mathbf{first}}^{*}$ – полное транзитивное замыкание отношения $\underline{\mathbf{first}}$,

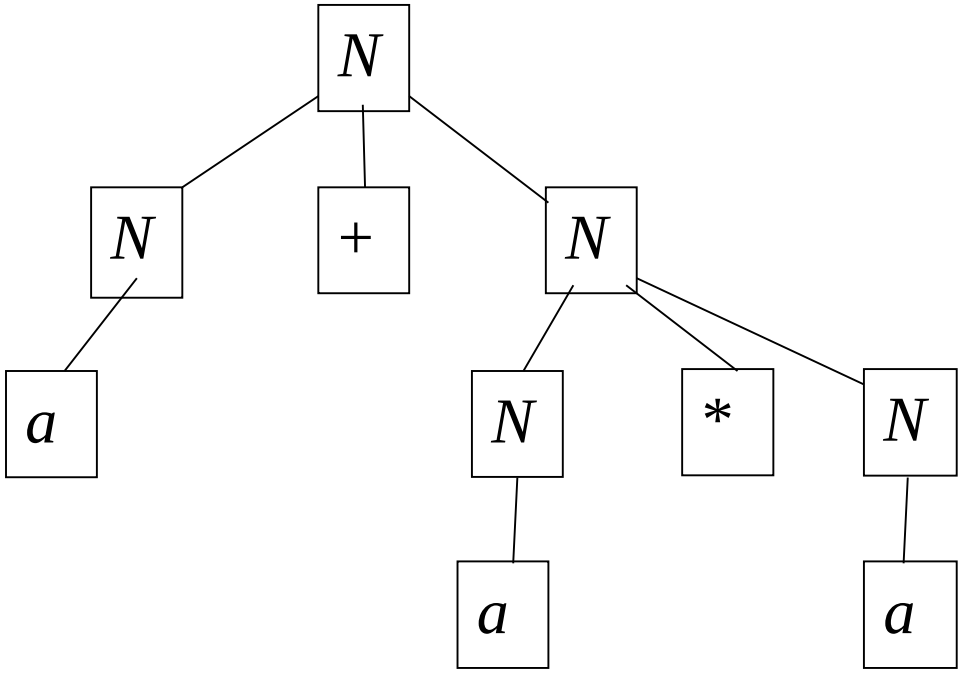
$\underline{\mathbf{last}}^{*}$ – полное транзитивное замыкание отношения $\underline{\mathbf{last}}$,

возведение в степень –1 обозначает вычисление обратного отношения, т.е. его транспонирование.

Пример. Анализ цепочки на входе: $a + a * a \perp$

№ шага	Вход	Магазин	Отношение	Правило
1	$a + a * a \perp$	$\perp$	$\perp < a$	$\leftarrow$
2	$+ a * a \perp$	$\perp a$	$a > +$	$N \rightarrow a$
6	$+ a * a \perp$	$\perp N$	$\perp < +$	$\leftarrow$
7	$a * a \perp$	$\perp N +$	$+ < a$	$\leftarrow$
8	$* a \perp$	$\perp N + a$	$a > *$	$N \rightarrow a$
10	$* a \perp$	$\perp N + N$	$+ < *$	$\leftarrow$
11	$a \perp$	$\perp N + N *$	$* < a$	$\leftarrow$
12	$\perp$	$\perp N + N * a$	$a > \perp$	$N \rightarrow a$
13	$\perp$	$\perp N + N * N$	$* > \perp$	$N \rightarrow N * N$
14	$\perp$	$\perp N + N$	$+ > \perp$	$N \rightarrow N + N$
16	$\perp$	$\perp N$		***Выход***

На рисунке изображено неявно строящееся при разборе дерево порождения цепочки: $a + a * a \perp$



LR(*k*)-анализатор

Детерминированный анализ снизу-вверх, при котором на каждом шаге анализируется не более ***k*** очередных символов во входной цепочке и такое количество верхних символов магазина, которое однозначно определяет действие – перенос или свертку.

LR(*k*)-грамматика: КС-грамматика, допускающая детерминированный LR(*k*)-анализ для некоторого *k*.

Для LR(1)-анализатора строится таблица, в которой столбцы помечены терминалами (включая $\perp$), а строки – одним или большим количеством терминальных и нетерминальных символов, которые могут оказаться на вершине магазина.

Пример.

Построение таблицы LR(1)-анализатора для грамматики простых арифметических выражений:

$$S \to S + T \mid T$$

$$T \to T * F \mid F$$

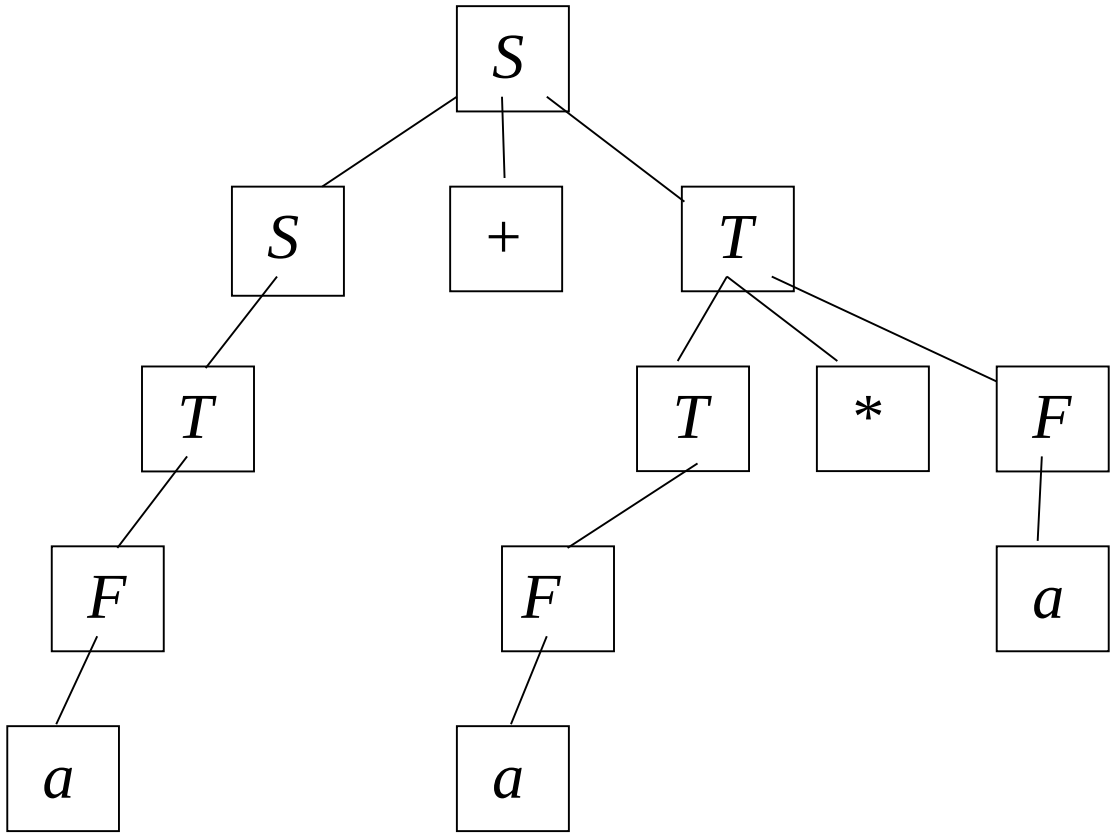
$$F \to (S) \mid a$$

Рассматриваются все такие возможные варианты, когда в верхней части магазина будет формироваться правая часть некоторого правила порождения.

	+	*	(	)	*a*	⊥
⊥			←		←	
+			←		←	
*			←		←	
(			←		←	
)	$F \to (S)$	$F \to (S)$		$F \to (S)$		$F \to (S)$
a	$F \to a$	$F \to a$		$F \to a$		$F \to a$
⊥ *S*	←					***допуск***
(*S*	←			←		
⊥ *T*	$S \to T$	←				$S \to T$
(*T*	$S \to T$	←		$S \to T$		
+ *T*	$S \to S + T$	←		$S \to S + T$		$S \to S + T$
⊥ *F*	$T \to F$	$T \to F$				$T \to F$
(*F*	$T \to F$	$T \to F$		$T \to F$		
+ *F*	$T \to F$	$T \to F$		$T \to F$		$T \to F$
* *F*	$T \to T * F$	$T \to T * F$		$T \to T * F$		$T \to T * F$

На рисунке изображено неявно строящееся LR(1)-анализатором дерево порождения цепочки:

$a + a * a.$



Трудоемкость алгоритмов грамматического анализа – линейная относительно *n* - количества терминальных символов в входной цепочке

LL(1)-анализатор: $O(k \cdot n)$,

где k – максимальное количество нетерминалов среди всех правых частей порождающих правил, причем таких нетерминалов, которые могут порождать λ.

Рекурсивный спуск,

грамматики простого предшествования,

LR(1)-анализатор: $O(m \cdot n)$,

где m – общее количество нетерминалов в грамматике.

Грамматики операторного предшествования: $O(2n)$.